PROJECT 1

Centralized School Learning Platform (COVID-19 | 2020)

Context & Motivation

In April 2020, due to COVID-19 lockdowns, my school abruptly transitioned to remote learning. Academic resources (PPTs, PDFs, assignments) were shared inconsistently via messaging platforms, leading to fragmentation, loss of materials, and minimal student–teacher interaction. With over 800 students, the school required a low-bandwidth, centralized, easy-to-use platform.

I led the end-to-end design, development, deployment, and maintenance of a centralized web-based learning platform using school-owned infrastructure.

Problem Statement

- **No centralized repository for learning materials**
- **Poor organization by class and subject**
- **No asynchronous doubt-resolution system**
- **Teachers had limited technical expertise**
- **Large user base required simplicity and reliability**

System Design & Architecture

Design Philosophy

- **Monolithic, server-rendered architecture**
- **Minimal dependencies (no cloud, no SPA frameworks)**

- **Optimized for low bandwidth and ease of maintenance**

Tech Stack (2020-Appropriate)

Layer	Technology
Frontend	**HTML5, CSS3, Bootstrap 4, Vanilla JS**
Backend	**Python (Flask)**
Database	**MySQL**
Authentication	**Session-based auth (Werkzeug hashing)**
Server	**Windows PC (School Lab)**
Web Server	**Apache (XAMPP)**
Hosting	**School subdomain + port forwarding**

Core Features

1. Role-Based Authentication

- **Roles: Student, Teacher, Admin**
- **Session-based login, password hashing**
- **Access control enforced at route level**

Scale Metrics

- **~800 students**
- **~45 teachers**
- **~3 admins**

---

2. Modular Academic Structure

- **Content organized by Class → Subject**
- **Reduced cognitive overload**
- **Improved navigation for non-technical users**

---

3. Learning Resource Management

- **Teachers upload PPTs, PDFs, assignments**
- **Files stored on server filesystem, indexed via MySQL**
- **Server-side rendering using Jinja templates**

Usage Metrics

- **~1,200+ resources uploaded (first 3 months)**
- **~300–400 daily student visits**

---

4. Admin Panel

- **Class & subject management**
- **Content moderation**
- **User access control**

UI intentionally minimal for teacher usability.

---

5. Student–Teacher Q&A Forum

- **Subject-wise question posting**
- **Teachers and senior students could respond**
- **Asynchronous interaction improved engagement**

Forum Metrics

- **~600+ questions**
- **~1,800+ answers (4 months)**
- **Response time reduced from days → hours**

Deployment & Hosting

Hosting Strategy

- **Self-hosted due to lack of cloud credits / LMS access**

Deployment Setup

- **Windows PC in school lab**
- **Apache + Flask backend**
- **Local MySQL database**
- **DNS mapped to school subdomain**
- **Router port forwarding for public access**

Operational Metrics

- **~95% uptime during school hours**
- **~200 concurrent users during exams**

Performance & Reliability

- **Server-side rendering reduced client load**
- **Bootstrap ensured mobile compatibility**
- **Indexed DB queries for frequent access**
- **Manual backups scripted**

Impact

- **Enabled uninterrupted learning for 800+ students**
- **Reduced reliance on third-party tools**
- **Improved academic continuity during lockdown**
- **Provided teachers a single, structured platform**

Limitations (Acknowledged)

- **Single-server deployment**
- **Manual scaling**
- **No real-time chat**
- **Limited UI interactivity**

SoP Reference – Key Reflections (Project 1)

Challenges Faced

- **First production system with real users**
- **Transition from academic coding to deployment**
- **Hosting without cloud infrastructure**
- **Designing for non-technical stakeholders**

Debugging & Problem Solving

- **File upload permission issues (Windows)**
- **Session persistence bugs**
- **DB normalization for redundancy reduction**
- **Load handling during peak usage**

> **Debugging shifted from fixing syntax errors to understanding user behavior, server constraints, and data integrity.**

Key Learnings

Technical

- **Backend–frontend integration**
- **Database design & normalization**
- **Auth & session management**
- **Server networking basics**

Non-Technical

- **Requirement gathering**
- **Usability-driven design**

- **Responsibility for production systems**

PROJECT 2

Autonomous GPS-Based Surveillance Drone

IETE Robotics Competition | 1st Prize (2019)

Overview

In 2019, during my final year of high school, I designed and built an autonomous aerial surveillance drone for the IETE Robotics Competition. The drone navigated predefined GPS waypoints, detected human presence using PIR sensors, and demonstrated actuator control via a payload release mechanism.

This project marked my first deep immersion into embedded systems, autonomy, and real-world uncertainty.

Problem Statement

- **Manual surveillance of large or remote areas is inefficient**
- **Need for autonomous navigation**
- **Human detection during patrol missions**
- **Low-cost, student-built solution**

System Architecture

`RC Failsafe`

```
    ↓
Flight Controller (Autopilot)
    ↓
GPS + IMU + Compass
    ↓
Autonomous Navigation Logic
    ↓
PIR Sensor (Human Detection)
    ↓
Telemetry & Logs
```

Technical Stack (2019 – Student Level)

Hardware

- **Flight Controller: APM 2.8 / Pixhawk 2.4.8**
- **Microcontroller: Arduino Nano**
- **GPS: u-blox NEO-6M**
- **PIR Sensor: HC-SR501**
- **Motors: 2200KV BLDC ×4**
- **ESCs: 30A**
- **Frame: F450**
- **Battery: 3S 5200mAh LiPo**
- **Telemetry: 433 MHz**
- **RC: FlySky FS-i6**
- **Payload Release: Servo-based mechanism**

Software

- **Firmware: ArduPilot (ArduCopter)**
- **Programming: Arduino C/C++**
- **Ground Station: Mission Planner**
- **Protocol: MAVLink**

Core Functionalities

Autonomous GPS Navigation

- **Waypoints defined in Mission Planner**
- **Autonomous takeoff, navigation, RTL**

Metrics

- **GPS accuracy: ±2–3 m**
- **Flight time: ~12–15 min**
- **Waypoints: 6–10 per mission**

Human Detection

- **PIR sensor monitored via Arduino**
- **Detection events logged during mission**

Metrics

- **Detection range: ~5–7 m**

- **Accuracy: ~80–85% (static tests)**

Safety & Fail-Safes

- **Manual RC override**
- **Low battery auto-land**
- **GPS RTL failsafe**
- **Geofencing**

Testing & Debugging

Key Challenges

- **GPS drift**
- **Sensor noise & false positives**
- **Power fluctuations**
- **Firmware configuration complexity**

Solutions

- **Compass calibration, vibration damping**
- **Debounce logic & sampling windows**
- **Conservative flight profiles**
- **Log-based PID tuning**

Results & Impact

- **Successfully demonstrated autonomous surveillance**
- **Logged human detection events**
- **Won 1st Prize – IETE Robotics Competition**
- **Individual project evaluated across institutions**

Key Metrics

- **Competition Rank: 1st**
- **Team Size: 1**
- **Autonomous Duration: 12–15 min**

SoP Reference – Key Reflections (Project 2)

Challenges Faced

- **Translating theory into airborne systems**
- **Debugging in uncontrolled environments**
- **Understanding control parameters**
- **Balancing autonomy with safety**

What I Learned

Technical

- **Embedded programming**
- **Sensor integration**

- **Autonomous navigation**
- **Control systems fundamentals**

Conceptual

- **Systems thinking**
- **Reliability & safety trade-offs**
- **Failure-driven learning**